Stanford CS329A：自我改进 AI Agent
Part 2：测试时计算扩展

Azalia Mirhoseini

AI Course Notes & Codex



课程：Stanford CS329A Self-Improving AI Agents（Autumn 2025）
本讲日期：2025-09-26　**公开视频发布日期**：2026-08-03
讲义制作日期：2026 年 8 月 10 日
频道 / 时长：Stanford Online　/ 1:03:20
视频链接：https://www.youtube.com/watch?v=-Ggc37xLj_Y
制作规范：wdkns/wdkns-skills（commit `39f1a04`）与本项目质量规范

目录

1 问题设定：模型不变，计算预算增加 2

1.1 四个评价量 . . . 2

1.2 本章小结 . . . 2

2 Large Language Monkeys：重复采样的 Scaling Law 2

2.1 生成与验证的两阶段系统 . . . 2

2.2 小模型为何能通过采样追上强模型 . . . 3

2.3 从单题指数曲线到数据集平均幂律 . . . 4

2.4 长尾是平均幂律的充分条件 . . . 6

2.5 计算分配范式的变化 . . . 6

2.6 本章小结 . . . 7

3 Generation–Verification Gap 7

3.1 Oracle coverage 是能力上界，不是可交付性能 . . . 7

3.2 为什么 majority vote 会失败 . . . 8

3.3 验证器应具备什么性质 . . . 9

3.4 本章小结 . . . 9

4 Compute-Optimal Test-Time Scaling 10

4.1 并行采样与串行修订 . . . 10

4.2 Outcome Reward Model 与 Process Reward Model . . . 11

4.3 混合宽度与深度 . . . 11

4.4 实验结论：最佳策略随难度变化 . . . 13

4.5 测试时 FLOPs 何时胜过预训练 FLOPs . . . 14

4.6 本章小结 . . . 14

5 Archon：搜索 Inference-Time Architecture 14

5.1 把推理系统视为可搜索架构 . . . 14

5.2 可组合操作 . . . 15

5.3 Fusion 为什么可能超过 oracle selection . . . 16

5.4 自然语言单元测试 . . . 16

5.5 深层推理架构与架构搜索 . . . 17

5.6 本章小结 . . . 19

6 工程实践：如何设计测试时扩展系统 19

6.1 先判断任务是否适合扩展 . . . 19

6.2 停止规则比最大预算更重要 19
6.3 可观测性要求 20
6.4 本章小结 20

7 拓展阅读 **20**

8 总结与延伸 **20**

1 问题设定：模型不变，计算预算增加

传统 scaling law 把主要计算投入训练阶段；test-time compute scaling 则问：**在模型参数固定以后，能否通过更多采样、搜索、修订和验证，提高同一个模型的最终成功率？**本讲围绕三组工作展开：

1. Large Language Monkeys：重复采样为什么呈现可预测的 scaling law；
2. Compute-optimal test-time scaling：并行采样和串行修订应如何分配预算；
3. Archon：如何自动搜索由 generator、critic、ranker、fuser 和 verifier 组成的推理架构。

本讲的核心分解

测试时扩展不是单一算法，而是三个问题：**生成**能否覆盖正确候选，**验证**能否识别正确候选，**预算控制**能否把有限计算投入最有价值的操作。只优化其中一个环节，端到端收益都可能被另外两个环节吞掉。

1.1 四个评价量

指标	含义
pass@1	系统只交付一个答案时的最终准确率
pass@k / coverage	k 个候选中至少存在一个正确答案的概率
best-of-N	生成 N 个候选后，由某个 verifier 选择一个答案
majority@N	按答案频率投票，隐含“正确答案最常出现”的假设

1.2 本章小结

增加推理计算首先扩大候选集，但最终系统只交付一个结果，因此 coverage 与 pass@1 必须分开测量。后续所有方法都可以看作在候选质量、选择质量和计算成本之间做权衡。

2 Large Language Monkeys：重复采样的 Scaling Law

2.1 生成与验证的两阶段系统

Repeated sampling 先从同一模型独立采样多个候选，再交给 verifier 选择。代码与形式化证明尤其适合这种方法，因为单元测试、编译器和 proof checker 能提供接近确定性的自动反馈。

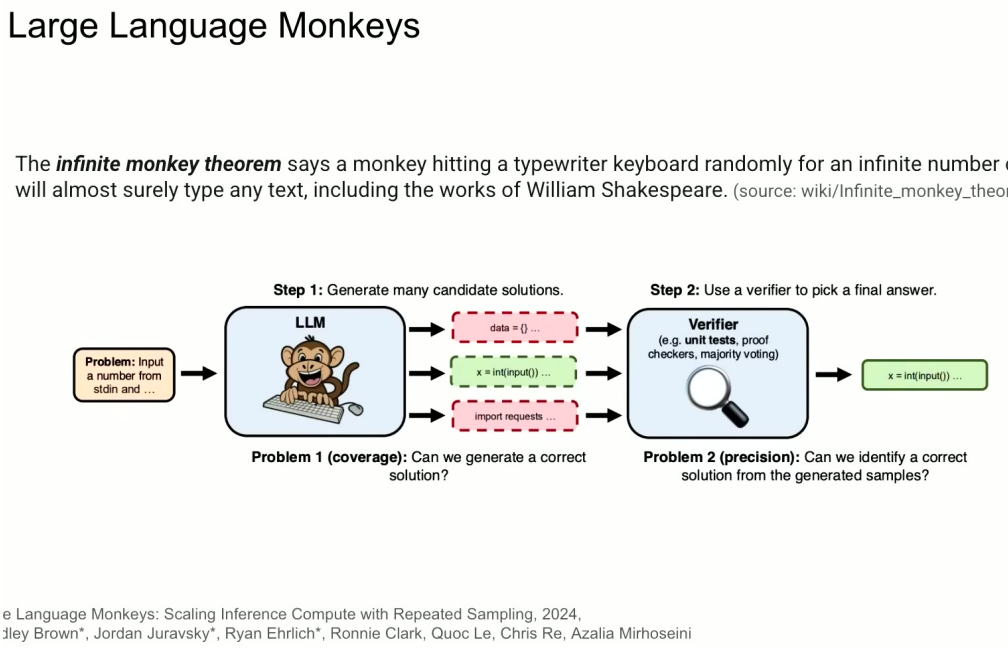


图 1: 多候选生成与验证构成 inference scaling 的基本流水线。[1]

假设问题 i 的单次采样成功概率为 p_i，独立采样 k 次的 oracle coverage 为：

$$C_i(k) = 1 - (1 - p_i)^k.$$

- p_i：模型对问题 i 的单次成功概率；
- k：采样数；
- $C_i(k)$：候选集合至少包含一个正确解的概率。

这说明每道题自身呈指数饱和：初始采样带来最大收益，之后边际增益逐步下降。

2.2 小模型为何能通过采样追上强模型

只要 $p_i > 0$，增加 k 就能提高 coverage。实验中，较小模型通过大量采样在数学、代码和形式化证明上显著缩小与强模型单次推理的差距。

[1] 视频讲解区间：00:01:18–00:02:22。

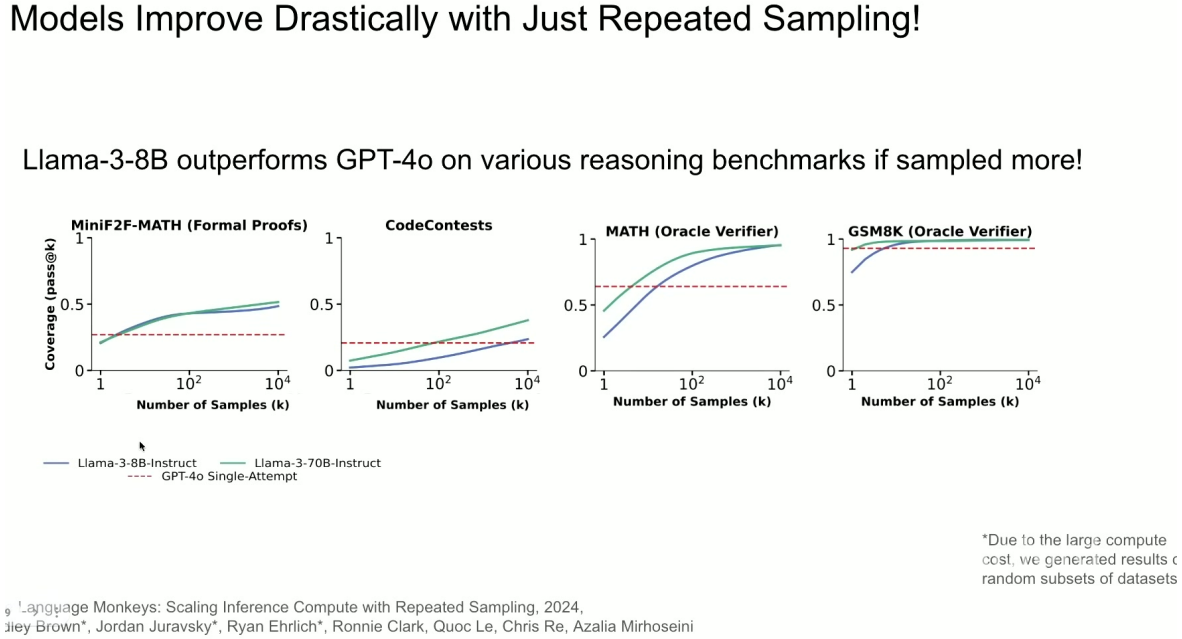


图 2: 重复采样在多个 reasoning benchmark 上显著放大模型能力。[2]

能力被“放大”还是被“发现”

Repeated sampling 并未改变参数，也没有让模型学会新算法。它做的是从模型已有的输出分布中，提高抽到低概率正确轨迹的机会。因此更准确的说法是**发现并利用潜在能力**，而不是创造全新能力。

2.3 从单题指数曲线到数据集平均幂律

课程给出的经验拟合是 exponentiated power law：

$$\bar{C}(k) \approx 1 - \exp(-ak^b), \qquad a > 0,\ 0 < b < 1.$$

- $\bar{C}(k)$：数据集上的平均 coverage；
- a：与模型和任务总体难度相关的尺度参数；
- b：控制 scaling 速度的指数；
- k：推理采样预算。

[2] 视频讲解区间：00:02:22–00:04:06。

Scaling Laws for Inference

- The relationship between coverage (accuracy) and the number of samples can be modelled using an *exponentiated power law*
- Similar to pretraining, this allows us to predict the performance as a function of scaling compute, but this time scaling happens only at inference time

$$c = \exp(ak^b)$$

k: number of samples
a & b: curve fitting parameters
c: coverage or fraction of problems solved by at least one sample

e Language Monkeys: Scaling Inference Compute with Repeated Sampling, 2024,
dley Brown*, Jordan Juravsky*, Ryan Ehrlich*, Ronnie Clark, Quoc Le, Chris Re, Azalia Mirhoseini

图 3: 平均 coverage 随采样数呈可预测的 exponentiated power law。[3]

单个问题是 $1-(1-p_i)^k$，为什么对问题平均后会出现幂律？关键在于 p_i 的分布：数据集中存在大量模型偶尔才能解出的难题，形成靠近零的长尾。

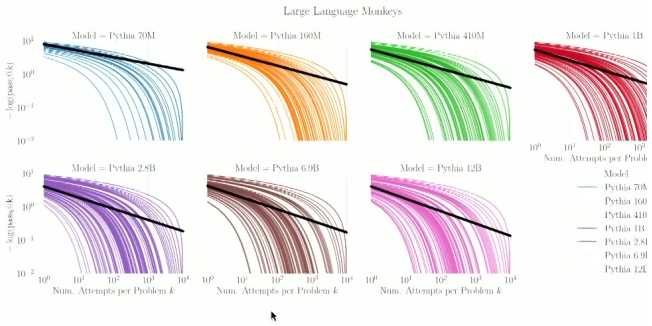
How Do Large Language Monkeys Get Their Power (Laws)?

For a SINGLE problem, we should expect -log(pass@k) to scale exponentially with k:

$$\begin{aligned}\text{pass}_\text{i}@\text{k} &= \mathop{\mathbb{E}}_{k \text{ Attempts}}\Big[1 - \mathbb{I}[\text{All } k \text{ Attempts Fail}]\Big] \\ &= 1 - \prod_{j=1}^{k} \mathop{\mathbb{E}}_{1 \text{ Attempt}}\Big[\mathbb{I}[j\text{-th Attempt Fails}]\Big].\end{aligned}$$

$$\text{pass}_\text{i}@\text{k} = 1 - (1 - \text{pass}_\text{i}@1)^k.$$

$$-\log(\text{pass}_\text{i}@\text{k}) \approx (1 - \text{pass}_\text{i}@1)^k.$$



-log(avg pass@k): Power Law (Polynomial) Scaling with k

-log(pass_i@k): Exponential Scaling with k

How can we reconcile these?

w Do Large Language Monkeys Get Their Power (Laws)?,
lan Schaeffer, Joshua Kazdan, John Hughes, Jordan Juravsky, Sara Price, Aengus Lynch, Erik Jones, Robert Kirk, Azalia Mirhoseini, Sanmi Koyejo,
ML 2025

图 4: 从小模型到大模型都观察到 inference scaling。[4]

[3] 视频讲解区间：00:05:18–00:07:06。
[4] 视频讲解区间：00:07:07–00:09:40。

2.4 长尾是平均幂律的充分条件

若问题成功概率在 $p \to 0$ 附近具有密度

$$f(p) \propto p^{\alpha-1},$$

则数据集平均失败率近似为：

$$1 - \bar{C}(k) = \mathbb{E}_p[(1-p)^k] \approx \int_0^\infty e^{-kp} p^{\alpha-1}\, dp \propto k^{-\alpha}.$$

- $f(p)$：不同问题的单次成功概率分布；
- α：靠近零点的长尾指数；
- $k^{-\alpha}$：随着采样预算增长而衰减的平均失败率。

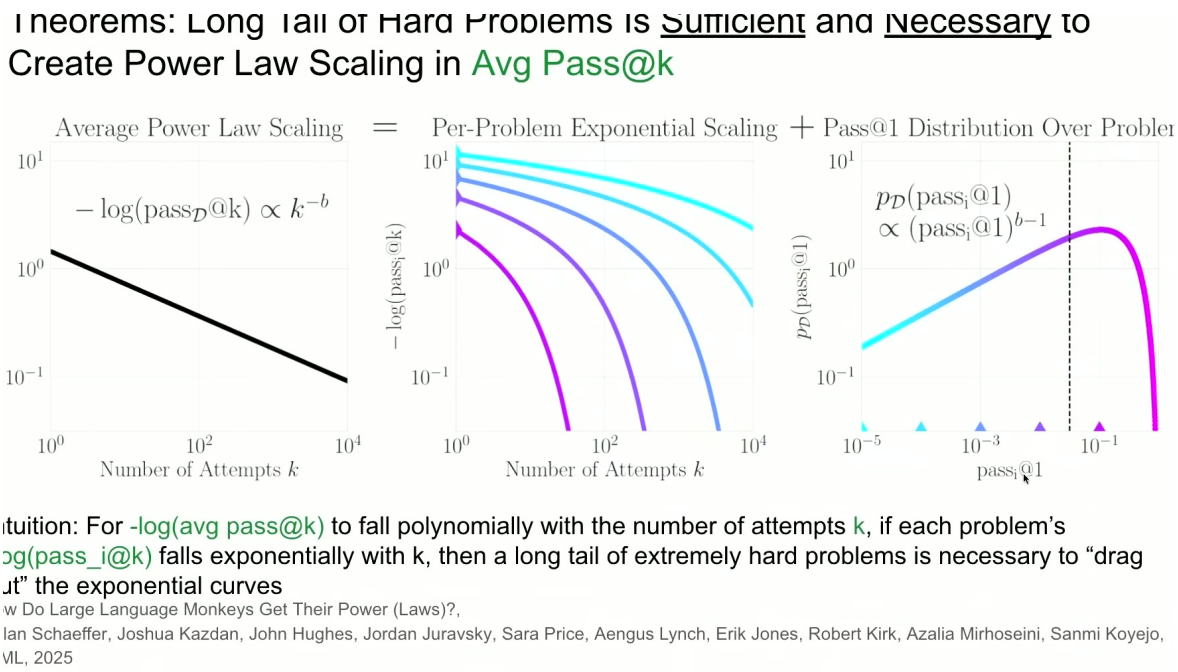


图 5: 大量低成功概率问题使数据集平均 pass@k 呈幂律扩展。[5]

长尾决定 inference scaling 的持续时间

如果所有题的 p_i 都较高，少量采样后就饱和；如果大量题的 p_i 极低但非零，收益会延续到很大的 k。因此推理 scaling 的形状不仅由模型决定，也由 benchmark 难度分布决定。

2.5 计算分配范式的变化

当推理计算可预测地换取成功率时，系统不必把所有预算都固化在预训练中。对少量高价值请求，投入大量 test-time compute 可能比训练并部署更大模型更经济。

[5] 视频讲解区间：00:09:40–00:11:42。

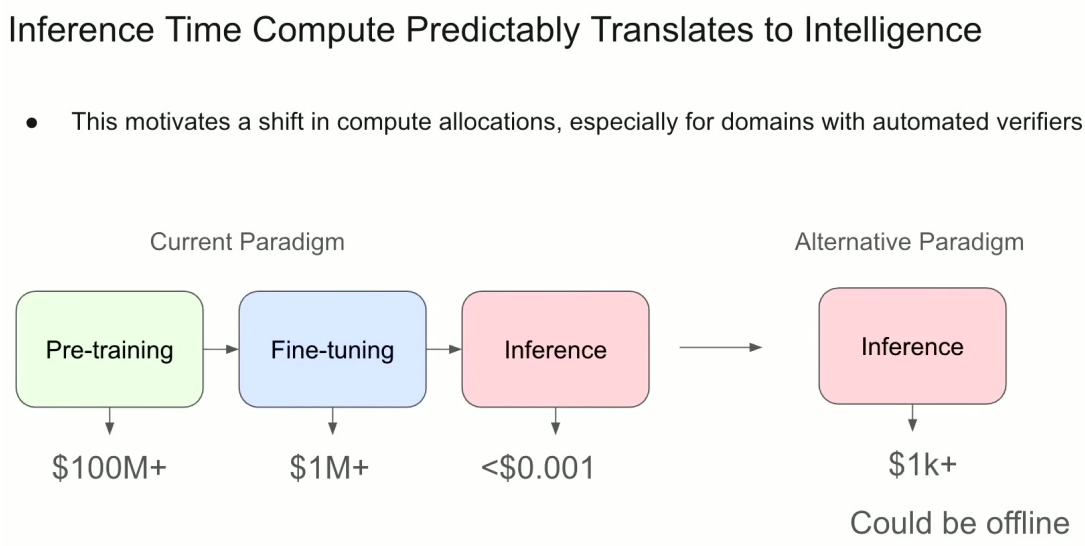


图 6: 从“训练大模型、单次推理”转向“模型适中、按请求投入推理计算”。[6]

2.6 本章小结

Repeated sampling 的收益来自问题成功概率的长尾。它对模型规模具有广泛适用性，但它证明的是 oracle coverage 可扩展；若没有可靠 verifier，正确候选仍可能无法转化为最终输出。

3 Generation–Verification Gap

3.1 Oracle coverage 是能力上界，不是可交付性能

Oracle verifier 知道 ground truth，因此总能从候选中挑出正确答案。真实系统通常只能使用多数投票、reward model 或 LLM judge，这些方法与 oracle 之间可能存在巨大差距。

[6] 视频讲解区间：00:11:42–00:12:38。

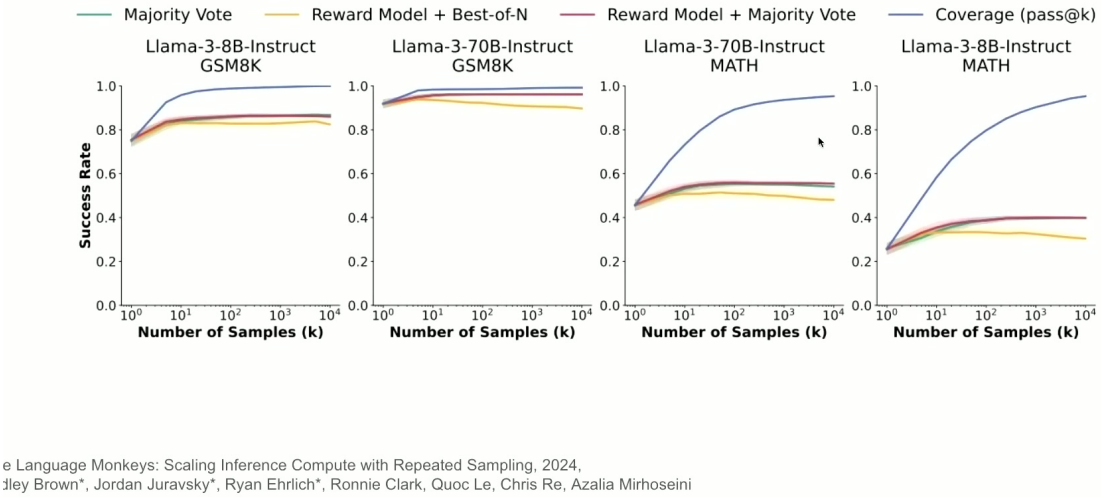


图 7: 常见选择方法远低于候选集合的 oracle coverage。[7]

定义：

$$G(k) = C_{\text{oracle}}(k) - A_{\text{selector}}(k),$$

- $C_{\text{oracle}}(k)$：候选集合中存在正确答案的比例；
- $A_{\text{selector}}(k)$：真实选择器交付正确答案的比例；
- $G(k)$：generation–verification gap。

3.2 为什么 majority vote 会失败

多数投票假设正确答案是分布的主模态。但对困难问题，模型可能产生许多相似错误，而正确轨迹极少出现。此时增加采样反而让错误模态的频率估计更稳定。

[7]视频讲解区间：00:15:32–00:18:44。

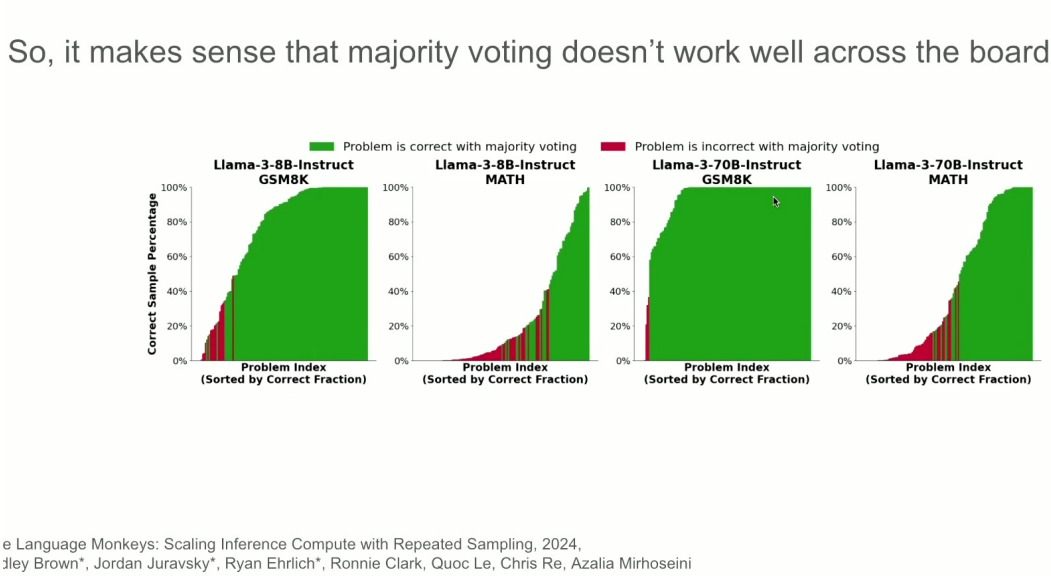


图 8：正确候选可能位于输出分布的稀有尾部。[8]

Self-consistency 依赖一个隐藏假设

多数投票只有在“正确推理路径比任一错误模式更集中”时才可靠。若模型系统性偏向同一种错误，self-consistency 会自信地选择错误答案。候选间的一致性不是正确性的充分证据。

3.3 验证器应具备什么性质

- **Soundness**：不要把明显错误判为正确；
- **Recall**：不要漏掉候选中的稀有正确解；
- **Calibration**：分数差异应反映真实成功概率；
- **Robustness**：对风格、长度和表面自信不过度敏感；
- **Cost**：验证成本应显著低于重新生成或人工检查。

可验证领域的结构优势

数学形式化证明和代码执行可把 verifier 外包给符号系统或真实环境。开放式写作、战略决策和科学假设则缺少便宜 ground truth，只能使用 learned reward model 或延迟反馈，因此更容易出现验证瓶颈。

3.4 本章小结

Inference scaling 的端到端上限由 verifier 决定。生成器越强、采样越多，候选分布越复杂，选择器反而可能面临更难的识别问题；因此 robust verification 是下一讲的核心主题。

[8] 视频讲解区间：00:18:44–00:19:44。

4 Compute-Optimal Test-Time Scaling

4.1 并行采样与串行修订

增加 test-time compute 有两种基本方式：

- **Parallel sampling**：从同一 prompt 独立生成多个候选，强调多样性；
- **Sequential revision**：让模型读取中间结果并修订，强调单轨迹深度。

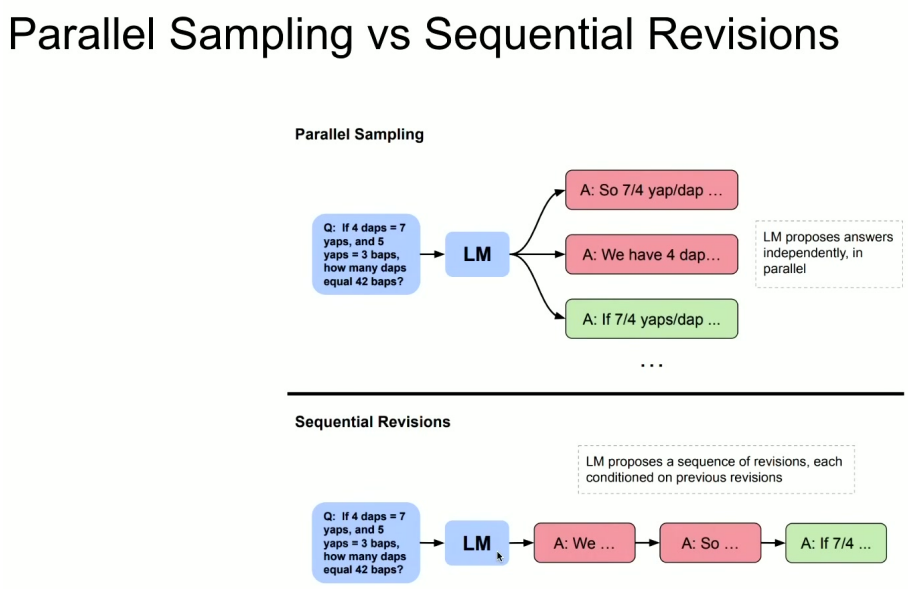


图 9: 并行采样用多个独立候选扩大搜索宽度。[9]

并行方法易于扩展，且错误不会跨候选传播；串行方法能利用反馈逐步改进，但若早期方向错误，后续步骤可能围绕错误前提继续优化。

[9] 视频讲解区间：00:27:00–00:29:06。

4.2 Outcome Reward Model 与 Process Reward Model

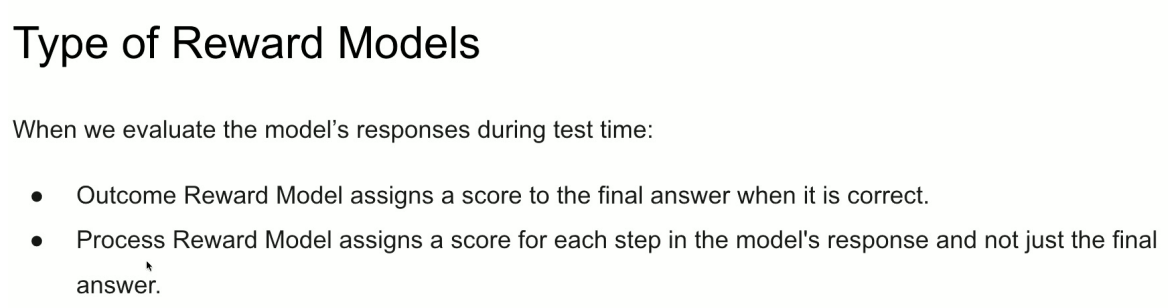


图 10: Outcome reward 与 process reward 提供不同粒度的搜索信号。[10]

- **ORM**（Outcome Reward Model）：只评价最终答案；适合 best-of-N，信用分配粗；
- **PRM**（Process Reward Model）：逐步评价推理过程；可用于 beam search 和提前剪枝。

若一条轨迹包含步骤 $z_{1:T}$，PRM 可提供局部得分 $r_t = r(z_t \mid z_{<t}, x)$，搜索器据此保留高潜力分支，而无需等到完整答案生成后才发现错误。

4.3 混合宽度与深度

Compute-optimal 策略不是固定选并行或串行，而是在总预算下组合两者：先探索多个初始候选，再对高分候选进行多轮修订。

[10] 视频讲解区间：00:29:06–00:30:08。

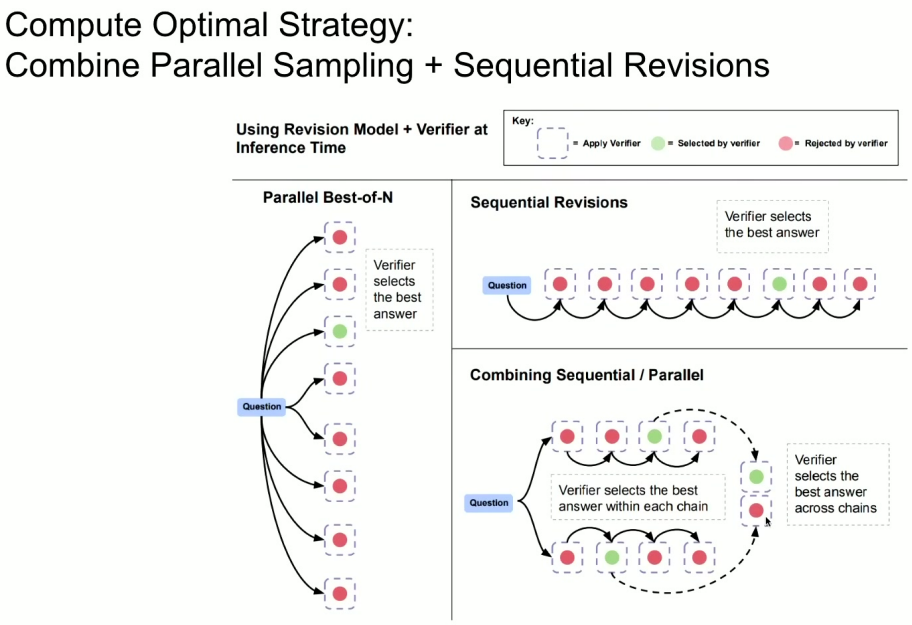


图 11: 并行采样扩大宽度，revision model 增加搜索深度。[11]

可写成预算约束：

$$B = n_{\text{parallel}} \cdot c_{\text{sample}} + n_{\text{revise}} \cdot c_{\text{revise}} + c_{\text{verify}}.$$

- B：单题总推理预算；
- n_{parallel}：并行候选数；
- n_{revise}：串行修订轮数；
- $c_{\text{sample}}, c_{\text{revise}}, c_{\text{verify}}$：各操作成本。

[11] 视频讲解区间：00:31:10–00:35:28。

4.4 实验结论：最佳策略随难度变化

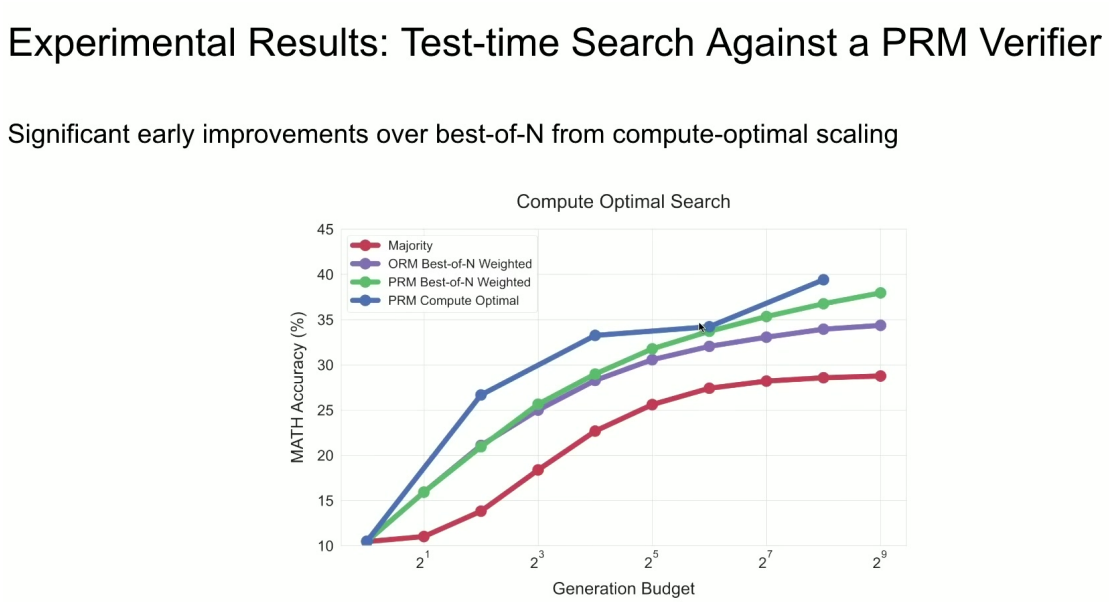


图 12: 在 PRM 指导下，compute-optimal search 早期收益高于纯 best-of-N。[12]

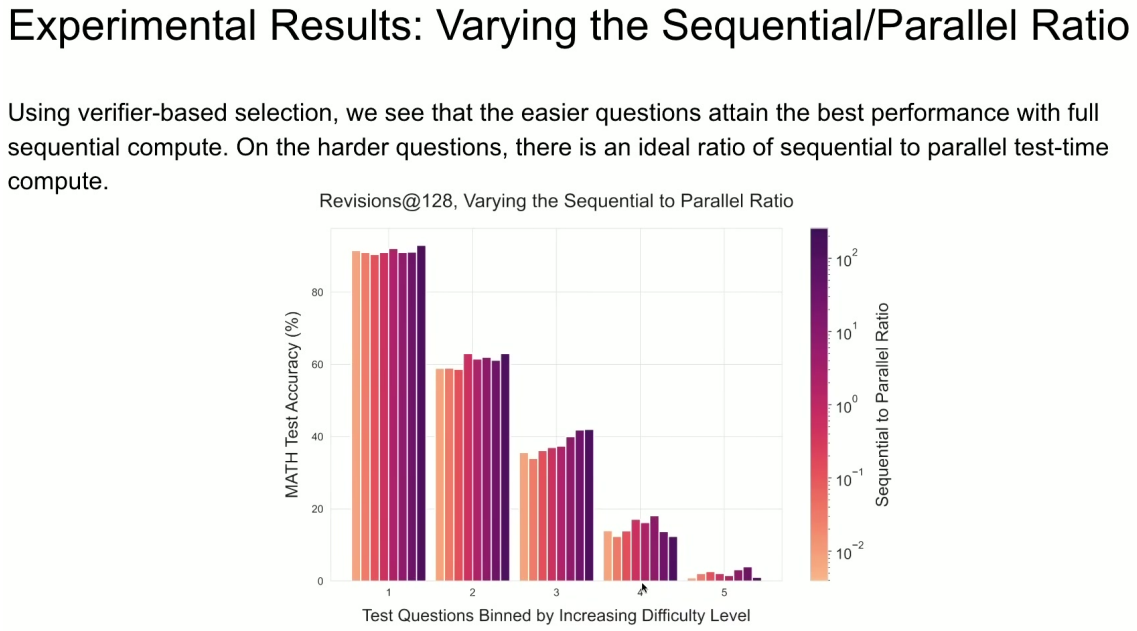


图 13: 简单题偏向串行修订，困难题需要一定并行探索。[13]

直觉是：

- 简单题通常已有接近正确的初始解，revision 能快速修正；

[12] 视频讲解区间：00:35:28–00:37:28。
[13] 视频讲解区间：00:37:28–00:40:12。

- 困难题的初始解可能落入完全错误的思路，需要并行探索不同方向；
- 过度串行会把预算浪费在修补不可救的轨迹；
- 过度并行则产生大量浅层候选，没有充分利用反馈。

4.5 测试时 FLOPs 何时胜过预训练 FLOPs

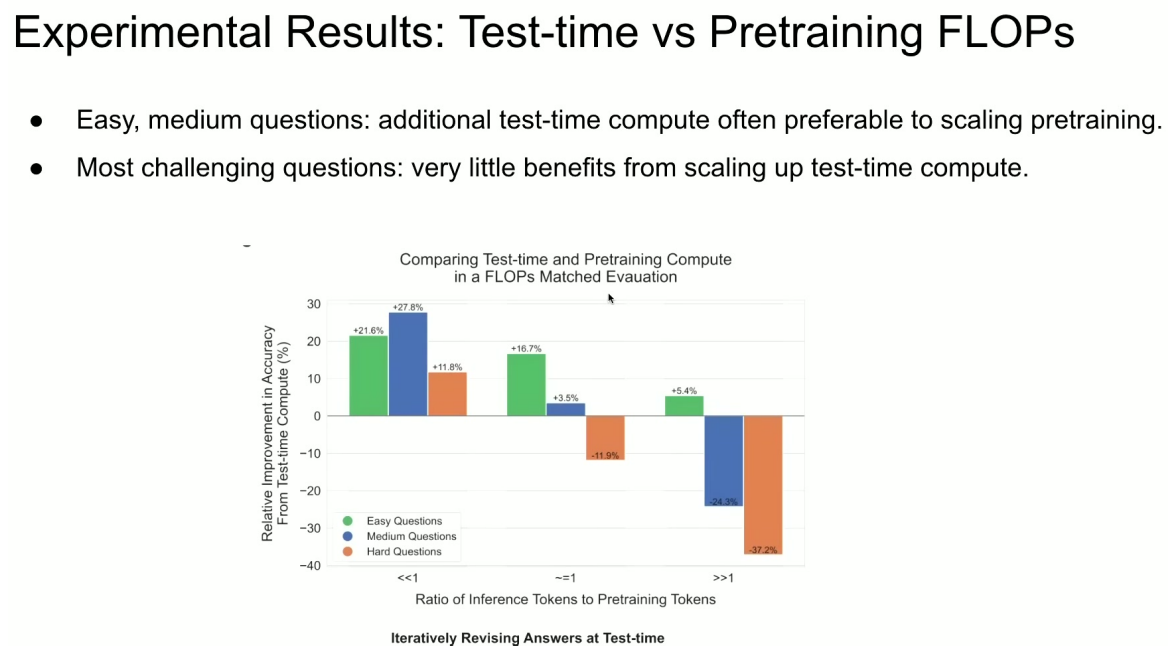


图 14: 中等难度问题常从 test-time compute 获益，而最难问题仍受基础能力限制。[14]

Test-time scaling 不是 pre-training scaling 的完全替代

当基础模型对某类问题几乎没有正确轨迹时，搜索无法放大不存在的信号。实验显示，对最难问题，增加推理计算的收益很小，扩大预训练或改进训练数据仍更有效。

4.6 本章小结

最优推理策略依赖题目难度、模型能力和 verifier 质量。真正的 compute-optimal 系统需要在线估计边际收益，动态决定继续采样、修订、验证还是停止。

5 Archon：搜索 Inference-Time Architecture

5.1 把推理系统视为可搜索架构

Archon 不再手工选择单一技巧，而是把 inference-time operations 当作神经网络中的层，搜索它们的组合、顺序、模型来源和调用次数。

[14] 视频讲解区间：00:40:12–00:42:32。

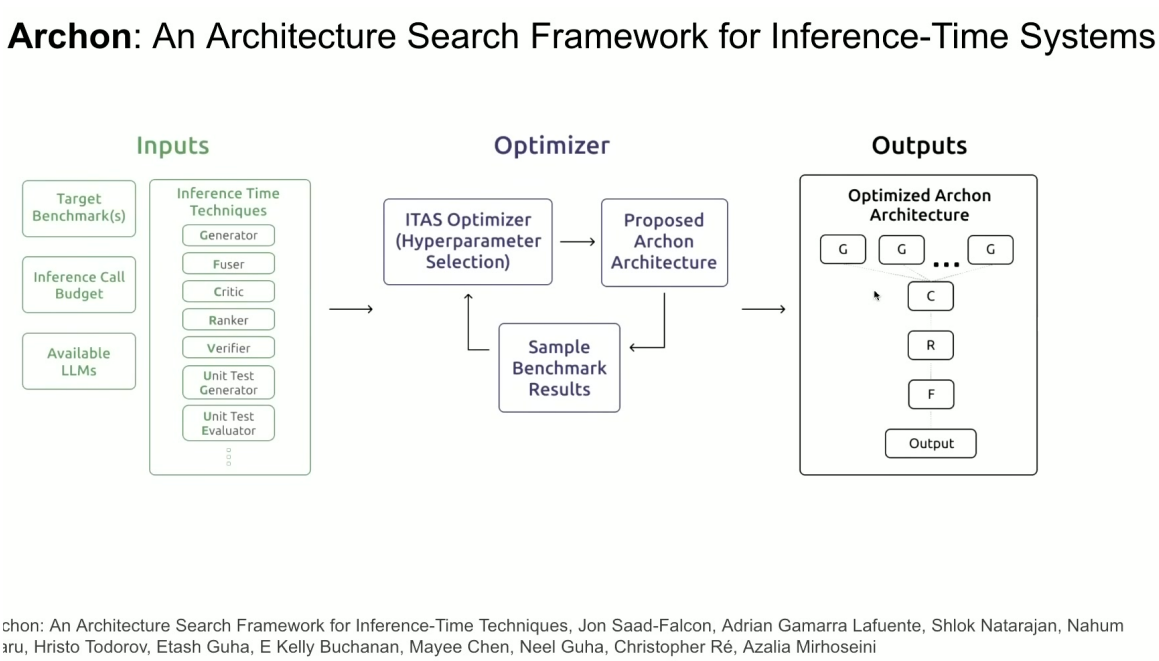


图 15: Archon 根据目标 benchmark 和可用模型搜索优化后的推理架构。[15]

其输入包括目标任务、候选模型、可用操作和成本约束；输出是一张执行图。例如多个 generator 并行采样，critic 识别缺陷，ranker 选择候选，fuser 综合答案，再进行下一轮批评与融合。

5.2 可组合操作

Inference Time Operations

Inference-time Technique	Definition	Input	Output	Inference Cost	Domains
Generator	Generates a candidate response from an instruction prompt	Instruction Prompt	Candidate Response(s)	1 call per cand.	All Domains
Fuser	Merges multiple candidate responses into a single response	Instruction Prompt + Candidate Response(s)	Fused Candidate Response(s)	1 call per cand.	All Domains
Critic	Generates strengths/weaknesses for each candidate response	Instruction Prompt + Candidate Response(s)	Candidate Response(s) Strengths/Weaknesses	1 call	All Domains
Ranker	Returns top-K candidate responses	Instruction Prompt + Candidate Response(s)	Ranked Candidate Response(s)	1 call	All Domains
Verifier	Returns the candidate responses with verified reasoning	Instruction Prompt + Candidate Response(s)	Verified Candidate Response(s)	2 calls per cand.	Reasoning Tasks
Unit Test Generator	Generates unit tests to evaluate the candidate responses	Instruction Prompt	Instruction Prompt + Unit Tests	1 call	Reasoning Tasks
Unit Test Evaluator	Uses generated unit tests to evaluate candidate response	Instruction Prompt + Unit Tests + Candidate Response(s)	Scored Candidate Response(s)	1 call per cand.	Reasoning Tasks

图 16: Archon 中常见 inference-time operation 的定义、输入、输出与成本。[16]

[15] 视频讲解区间：00:45:47–00:48:36。

[16] 视频讲解区间：00:48:36–00:51:20。

操作	功能	主要风险
Generator	产生一个或多个候选	缺乏多样性、重复错误
Critic	指出候选缺陷与遗漏	批评不准确或只关注表面形式
Ranker	对候选排序	位置偏差、长度偏差、judge 偏差
Fuser	综合多个候选的互补部分	把错误也融合进去
Verifier / Tests	检查约束与正确性	测试覆盖不足、奖励投机

5.3 Fusion 为什么可能超过 oracle selection

Oracle selection 只能从现有候选中挑一个完整答案；fuser 可以从多个不完整候选中提取互补信息，生成一个原候选集合中不存在的新答案。

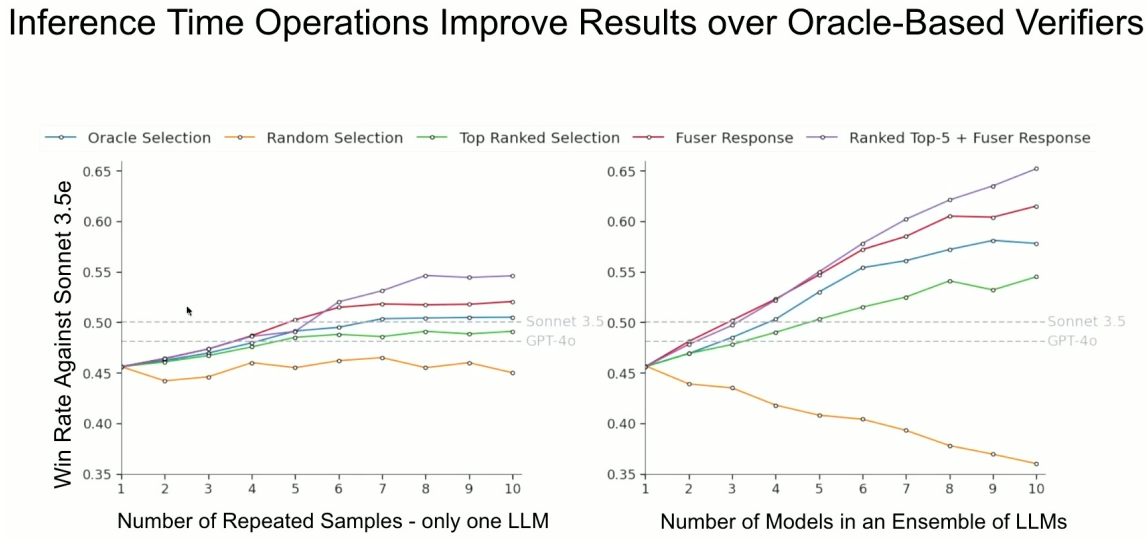


图 17: Rank-then-fuse 在部分任务上优于只选择单一候选。[17]

“超过 oracle”并不违反定义

这里的 oracle 只是在原始候选中选择正确答案；fusion 会执行新的模型调用并生成新候选，因此它改变了候选空间。它不是一个更强的选择器，而是额外的生成步骤。

5.4 自然语言单元测试

对于缺少可执行 ground truth 的任务，Archon 让模型生成自然语言测试或约束，再用它们检查候选回答。这相当于先把模糊目标分解为可局部验证的规格。

[17] 视频讲解区间：00:51:20–00:54:58。

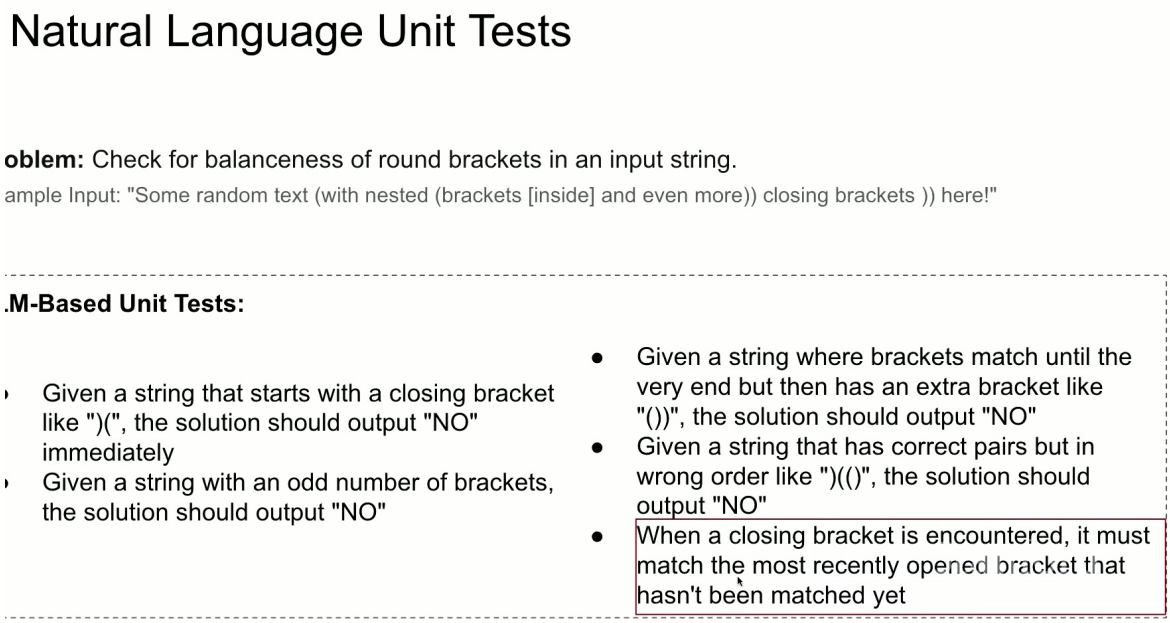


图 18: 自然语言 unit tests 把任务要求转换成可检查条件。[18]

这种做法提升了结构化验证能力，但测试由模型生成，仍可能遗漏关键需求。最稳妥的使用方式是把用户明确要求、领域规则和模型生成测试合并，而不是完全依赖自生成规格。

5.5 深层推理架构与架构搜索

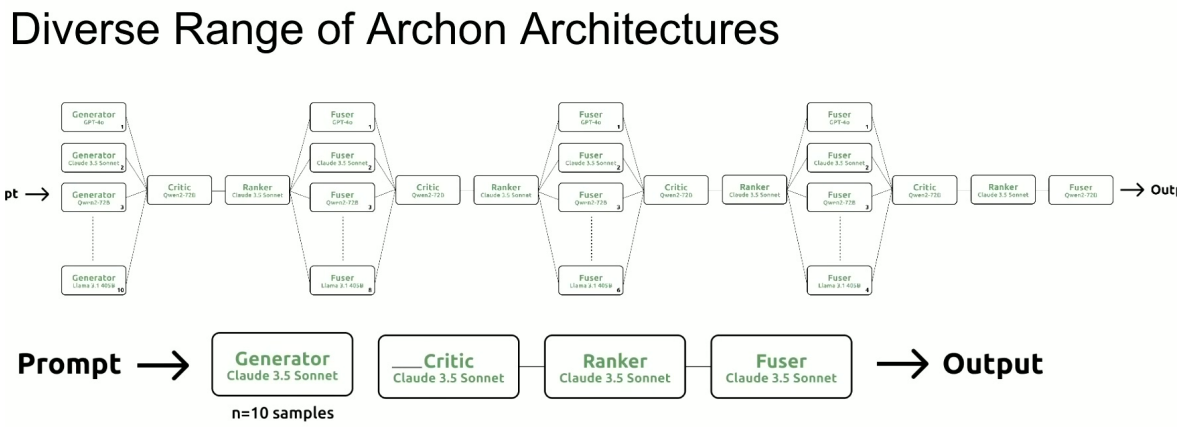


图 19: Archon 搜索出的多层 generator–critic–ranker–fuser 结构。[19]

[18] 视频讲解区间：00:55:30–00:56:24。

[19] 视频讲解区间：00:56:24–00:59:32。

架构空间会随层数、操作类型、模型选择和超参数组合爆炸。Archon 使用贝叶斯优化，以较少试验预测哪些配置最可能提升目标指标。

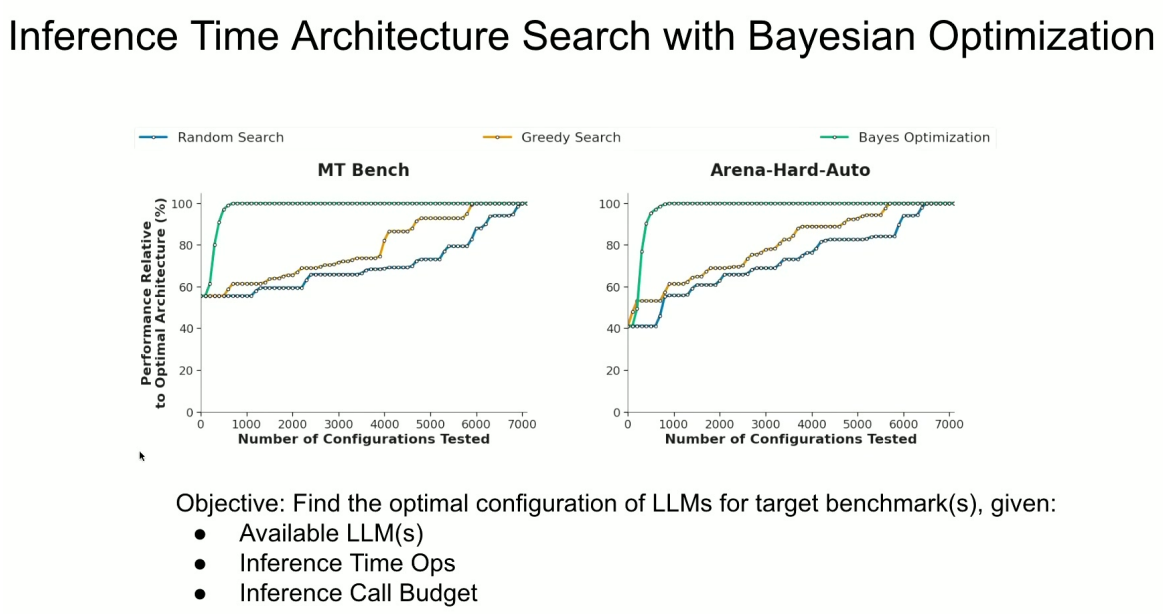


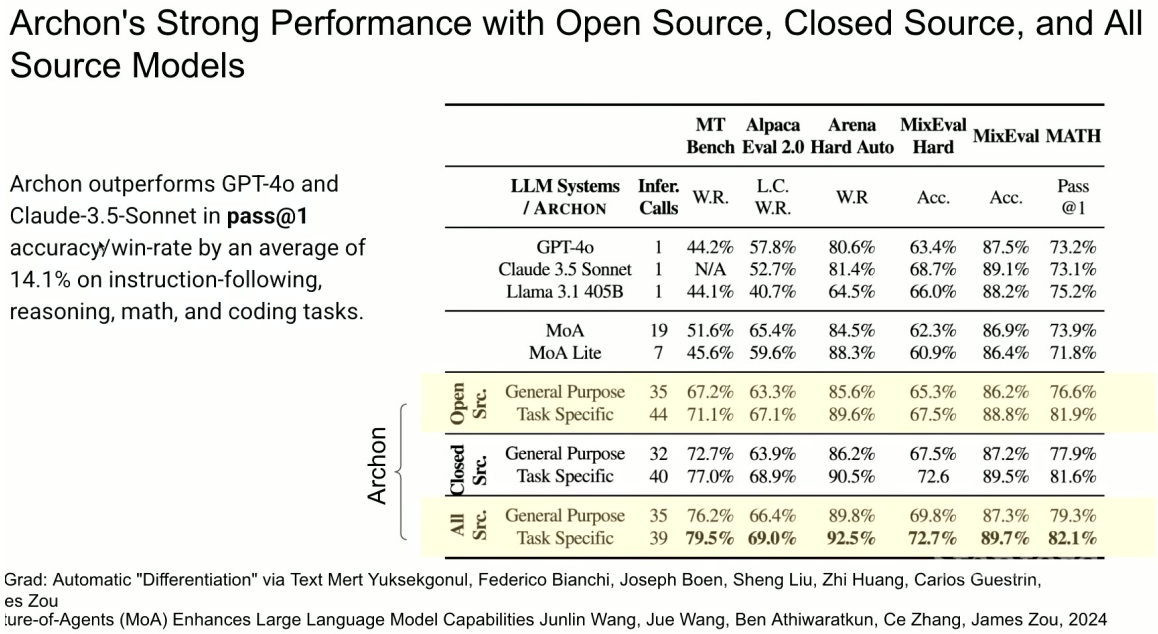
图 20: 贝叶斯优化比随机或贪心搜索更高效地发现高性能架构。[20]

若配置为 a，目标函数可写成多目标效用：

$$J(a) = \text{Quality}(a) - \lambda_c \text{Cost}(a) - \lambda_l \text{Latency}(a).$$

- $\text{Quality}(a)$：目标 benchmark 上的准确率或偏好分；
- $\text{Cost}(a)$：模型调用和 token 成本；
- $\text{Latency}(a)$：关键路径延迟；
- λ_c, λ_l：部署场景对成本与延迟的权重。

[20] 视频讲解区间：00:59:32–01:01:20。



	LLM Systems / ARCHON	Infer. Calls	MT Bench W.R.	Alpaca Eval 2.0 L.C. W.R.	Arena Hard Auto W.R	MixEval Hard Acc.	MixEval Acc.	MATH Pass @1
	GPT-4o	1	44.2%	57.8%	80.6%	63.4%	87.5%	73.2%
	Claude 3.5 Sonnet	1	N/A	52.7%	81.4%	68.7%	89.1%	73.1%
	Llama 3.1 405B	1	44.1%	40.7%	64.5%	66.0%	88.2%	75.2%
	MoA	19	51.6%	65.4%	84.5%	62.3%	86.9%	73.9%
	MoA Lite	7	45.6%	59.6%	88.3%	60.9%	86.4%	71.8%
Open Src.	General Purpose	35	67.2%	63.3%	85.6%	65.3%	86.2%	76.6%
Open Src.	Task Specific	44	71.1%	67.1%	89.6%	67.5%	88.8%	81.9%
Closed Src.	General Purpose	32	72.7%	63.9%	86.2%	67.5%	87.2%	77.9%
Closed Src.	Task Specific	40	77.0%	68.9%	90.5%	72.6	89.5%	81.6%
All Src.	General Purpose	35	76.2%	66.4%	89.8%	69.8%	87.3%	79.3%
All Src.	Task Specific	39	**79.5%**	**69.0%**	**92.5%**	**72.7%**	**89.7%**	**82.1%**

图 21: Archon 在开源、闭源与混合模型池上取得稳定提升。[21]

5.6 本章小结

Archon 把 “prompt engineering” 提升为系统架构搜索：不只优化一句 prompt，而是优化多个模型调用之间的数据流、控制流和成本。其局限是搜索结果可能过拟合 benchmark，且复杂架构的延迟、可解释性和维护成本显著提高。

6 工程实践：如何设计测试时扩展系统

6.1 先判断任务是否适合扩展

1. 基础模型是否偶尔能产生正确答案，即 $p > 0$?
2. 是否存在比生成更便宜、更可靠的 verifier?
3. 请求价值是否足以覆盖额外 token、延迟和能源?
4. 候选能否并行，还是必须串行读取环境状态?
5. 失败动作是否可逆，能否安全重试?

6.2 停止规则比最大预算更重要

系统可在以下条件提前停止：

- verifier 分数超过可信阈值；

[21] 视频讲解区间：01:01:20–01:03:02。

- 多个独立 verifier 达成一致；
- 新候选的边际质量提升低于成本；
- 连续修订没有改变关键错误；
- 达到延迟、费用或安全上限。

在线预算控制器

理想系统不是固定 best-of-64，而是先用小预算估计问题难度与候选分布，再决定是否扩展。简单题快速返回，困难但可验证的题投入更多计算，模型完全不会的题及时升级到更强模型或人工处理。

6.3 可观测性要求

应记录每个候选的生成模型、随机种子、token 数、验证分数、选择理由、工具结果和停止原因。否则只能看到最终答案，无法区分生成失败、验证失败、搜索策略失败和预算不足。

6.4 本章小结

Test-time scaling 是一个控制系统问题。可交付实现需要明确候选空间、验证信号、预算函数和停止规则，并用端到端 pass@1、成本和延迟共同评价。

7 拓展阅读

- Brown et al., *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*
- *How Do Large Language Monkeys Get Their Power (Laws)?*
- Snell et al., *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*
- Saad-Falcon et al., *Archon: An Architecture Search Framework for Inference-Time Techniques*

8 总结与延伸

本讲建立了测试时计算扩展的完整图景。Repeated sampling 通过长尾成功概率获得可预测的 coverage scaling；generation–verification gap 说明候选覆盖并不自动变成最终准确率；compute-optimal scaling 通过并行探索和串行修订的动态配比提高单位计算收益；Archon 则把 generator、critic、ranker、fuser 和 verifier 的组合提升为可搜索的系统架构。

可以把最终性能近似理解为：

$$\text{Delivered Quality} = \text{Generation Coverage} \times \text{Selection Reliability} \times \text{Budget Efficiency}.$$

三者中任一项接近零，额外推理计算都会失效。下一讲因此聚焦 robust verification：如何训练 outcome verifier 与 process verifier，如何自动构造步骤级监督，以及如何用多个弱 verifier 缩小选择差距。